Optimierungsverfahren

Übungsblatt 7

Übungsblatt 7 — 6. Metaheuristiken

Aufgabe 1 — Inspiration vs. Substanz *(10 Punkte)*

a) Erkläre kurz die natürliche Inspiration der drei Metaheuristiken **Simulated Annealing**, **Particle Swarm Optimization** und **Ant Colony Optimization** und nenne für jede ein typisches Anwendungsgebiet, das sich aus der Inspiration logisch ergibt. *(6 P)*

b) Im Skript steht der Hinweis: *„Analogie als Inspiration ok; als Selbstzweck schlecht (verdeckte Wiedererfindung).“* Erläutere, was damit gemeint ist, und gib ein nachvollziehbares Beispiel: Eine Forscherin „erfindet“ den Algorithmus *Glow-Worm Search*, bei dem Glühwürmchen sich gegenseitig anziehen und der hellste Punkt das Optimum markiert. Begründe, welche bereits im Kapitel behandelte Metaheuristik damit faktisch wiedererfunden wird und woran man das erkennt. *(4 P)*

Lösung:

a)

- **Simulated Annealing** — Inspiration: kontrolliertes Abkühlen von Metall (Annealing). Bei hoher Temperatur T werden viele/auch verschlechternde Änderungen akzeptiert, bei langsamer Abkühlung stabilisiert sich das System in einem (möglichst tiefen) Energiezustand. Anwendung: kombinatorische Optimierung mit vielen lokalen Minima (z. B. VLSI-Layout, TSP).
- **PSO** — Inspiration: Schwarmverhalten (Vögel/Fische); Partikel orientieren sich an eigener und globaler Best-Position, gute Lösungen ziehen schlechte an. Anwendung: kontinuierliche Parameter-Tuning-Probleme (z. B. Reglerauslegung).
- **ACO** — Inspiration: Pheromonpfade von Ameisen — kürzere Pfade werden öfter benutzt, sammeln mehr Pheromon, werden positiv verstärkt. Anwendung: **Routing-Probleme** (TSP, Netzwerk-Routing, Vehicle Routing).

b) Gemeint ist, dass eine biologische/physikalische Analogie nur dann Mehrwert hat, wenn sie zu einem strukturell *neuen* algorithmischen Mechanismus führt. Wird sie nur sprachlich umgehängt, entsteht eine „Wiedererfindung“ — derselbe Suchmechanismus mit neuem Etikett. Im Beispiel: Glühwürmchen ziehen sich gegenseitig an → eine Population von Lösungen, jede orientiert sich an einer „helleren“ (= besseren f-Wert) Lösung im Suchraum. Das ist mechanisch identisch zu **PSO** (Anziehung jedes Partikels an global/lokal beste Lösungen). Erkennungsmerkmal: Population, Bewertung per Zielfunktion, Bewegung in Richtung der besten Mitglieder, kein qualitativ neuer Mechanismus (kein Pheromon, keine Selektions-/Mutationskette, keine Temperatur).

Aufgabe 2 — 1+1-ES per Hand *(20 Punkte)*

Du minimierst $f(x_1, x_2) = x_1^2 + x_2^2$ mit der **1+1-ES**.

- Startlösung: $\vec{x} = (2{,}0;\ 2{,}0)$, also $y = f(\vec{x}) = 8{,}0$
- $\vec{\sigma}_0 = (1{,}0;\ 1{,}0)$
- Speicherlänge $g = 5$, Multiplikator $a = 1{,}5$
- Vorgegebene Mutationen $\vec{\epsilon}^{(t)}$ in den nächsten 5 Generationen (bereits aus $\mathcal{N}(0, \sigma_i)$ gezogen, in Einheiten von σ, d. h. tatsächlicher Schritt $= \sigma_i \cdot \epsilon_i$):

t	ϵ_1	ϵ_2
1	−0,8	−0,5
2	+1,2	−0,3
3	−0,4	−0,6
4	+0,9	+1,1
5	−0,2	−0,4

a) Führe die 5 Generationen Schritt für Schritt durch. Trage in eine Tabelle für jedes t ein: $\vec{x}_{\text{next}}$, y_{next}, akzeptiert? (1 / 0). $\vec{\sigma}$ wird in dieser Aufgabe (a) noch **nicht** angepasst. *(10 P)*

b) Nach diesen 5 Generationen ist G vollständig gefüllt. Wende die **1/5-Rule** einmal an: berechne $s = \sum G/g$ und passe $\vec{\sigma}$ entsprechend an. Gib das neue $\vec{\sigma}$ an. *(5 P)*

c) Begründe mit dem Skript-Hinweis, warum dieselbe 1+1-ES auf der **Rastrigin-Funktion** (siehe CMA-ES-Abschnitt) deutlich schlechter abschneiden würde — auch dann, wenn man a und g sorgfältig wählt. Welche Eigenschaft von Rastrigin verletzt die Annahme der 1/5-Rule? *(5 P)*

Lösung:

a) Tatsächlicher Mutationsschritt: $\Delta x_i = \sigma_i \cdot \epsilon_i$. Da $\vec{\sigma} = (1, 1)$ konstant bleibt, ist $\Delta x_i = \epsilon_i$.

t	$\vec{x}_{\text{next}}$	y_{next}	besser?	akzeptiert	$\vec{x}$ danach	y danach
1	(1,2; 1,5)	$1{,}44 + 2{,}25 = 3{,}69$	ja (vs. 8,0)	**1**	(1,2; 1,5)	3,69
2	(2,4; 1,2)	$5{,}76 + 1{,}44 = 7{,}20$	nein (vs. 3,69)	**0**	(1,2; 1,5)	3,69
3	(0,8; 0,9)	$0{,}64 + 0{,}81 = 1{,}45$	ja	**1**	(0,8; 0,9)	1,45
4	(1,7; 2,0)	$2{,}89 + 4{,}00 = 6{,}89$	nein	**0**	(0,8; 0,9)	1,45
5	(0,6; 0,5)	$0{,}36 + 0{,}25 = 0{,}61$	ja	**1**	(0,6; 0,5)	0,61

Speicher $G = [1, 0, 1, 0, 1]$.

b) $s = (1 + 0 + 1 + 0 + 1)/5 = 3/5 = 0{,}6$. Da $s = 0{,}6 > 1/5$, gilt $\vec{\sigma} \cdot= a = 1{,}5$:

$$\vec{\sigma}_{\text{neu}} = (1{,}0;\ 1{,}0) \cdot 1{,}5 = (1{,}5;\ 1{,}5)$$

Interpretation: Erfolge sind häufig → Schritt war zu klein → vergrößern.

c) Das Skript hält explizit fest: *„1/5-Rule: theoretisch optimal für Kugelfunktion $\sum x_i^2$, $n \to \infty$; nicht für multimodale Probleme.“* Rastrigin $f(\vec{x}) = 10n + \sum(x_i^2 - 10\cos(2\pi x_i))$ ist hochgradig **multimodal** — die Cosinus-Terme erzeugen ein dichtes Gitter lokaler Minima. Die 1/5-Rule kalibriert σ rein aus der lokalen Erfolgsrate auf einer (näherungsweise) konvexen, glatten Fitnesslandschaft. In der Rastrigin-Landschaft entsteht aber dauerhaft eine *hohe* Erfolgsrate **innerhalb** eines lokalen Trichters → die Regel

verkleinert/stabilisiert σ und die 1+1-ES bleibt im erstbesten lokalen Minimum hängen. Es fehlt der Mechanismus, σ groß genug zu halten, um Trichter zu *verlassen* — genau die Lücke, die CMA-ES (Restarts/große Population, adaptierte Kovarianz) füllt.

Aufgabe 3 — Kodierungen & Operatoren auf neuem Problem *(15 Punkte)*

Gegeben sei das Problem **„Stundenplan-Slot-Belegung“**: 8 Lehrveranstaltungen $L_1, \ldots, L_8$ sollen je entweder im Vormittagsslot (0) oder Nachmittagsslot (1) stattfinden. Die Fitness f misst die Anzahl der Hörsaal-Konflikte (zu minimieren).

a) Welche **Kodierung** aus dem Kapitel ist hier am natürlichsten? Begründe und gib einen konkreten Lösungskandidaten als $\vec{g}$ an. *(3 P)*

b) Gib einen passenden **Mutationsoperator** und einen passenden **Rekombinationsoperator** für diese Kodierung an. Wende beide einmal exemplarisch an: nimm

$\vec{g}_1 = 10110010$ und $\vec{g}_2 = 01101001$, Crossover-Punkt nach Position 3, Mutation flippt anschließend Bit 6 des ersten Kindes. Gib $\vec{g}_1', \vec{g}_2'$ und das mutierte Kind an. *(6 P)*

c) Begründe, warum eine **kontinuierliche** Kodierung mit Mutation $\vec{x} + \vec{\epsilon}$, $\vec{\epsilon} \sim \mathcal{N}(0, \sigma)$, hier ungeeignet wäre. Welcher Schritt würde brechen? *(3 P)*

d) Auf welche Anwendungs-Familie aus dem Skript fällt das Problem? *(3 P)*

Lösung:

a) **Binäre Kodierung** (Bitstring der Länge 8). Jede Variable ist binär (Slot 0 oder 1), die Codierung bildet den Lösungsraum 1:1 ab. Beispielkandidat: $\vec{g} = 10110010$ — d. h. L_1, L_3, L_4, L_7 nachmittags, der Rest vormittags.

b)

- Mutation: **single bit flip** (ein zufälliges Bit kippen).
- Rekombination: **single point crossover**.

Anwendung mit Crossover nach Position 3:

```
g1 = 101 | 10010
g2 = 011 | 01001
```

Kinder:

- $\vec{g}_1' = 101\,01001$
- $\vec{g}_2' = 011\,10010$

Mutation Bit 6 von $\vec{g}_1'$ (Position 6 ist die 6. Stelle, aktuell 0):

$$\vec{g}_1'' = 10101101$$

c) Eine kontinuierliche Kodierung würde reelle Werte $x_i \in \mathbb{R}$ liefern; die Slot-Zuordnung ist aber **diskret-binär**. Der Mutationsoperator $\vec{x} + \vec{\epsilon}$ mit normalverteiltem Rauschen erzeugt Werte wie $0{,}37$ oder $-1{,}2$, die keiner Slot-Belegung entsprechen — die Zielfunktion f (Konflikte) ist auf solchen Werten gar nicht definiert. Es bräuchte eine künstliche Diskretisierungsabbildung, die genau die Information zerstört, die die Mutation erzeugt hat (kleine Änderungen flippen kein Bit, große Änderungen wirken zufällig).

d) **Termin-/Maschinenplanung** (Skript-Liste der Anwendungen).

Aufgabe 4 — Tuning vs. Self-Adaptation vs. Parameter Control *(15 Punkte)*

Ein Team optimiert mit einer EA das aerodynamische Profil einer Düse (vgl. Schwefels frühe Arbeit). Es stehen drei Hyperparameter zur Debatte: Populationsgröße μ, Mutationsschrittweite σ und Mutationswahrscheinlichkeit p_m.

a) Ordne den drei im Skript genannten Tuning-Ansätzen (**Tuning**, **Self-adaptation**, **Parameter Control**) jeweils ein konkretes Vorgehen für *einen* der Hyperparameter zu, das für diesen Ansatz typisch ist. *(6 P)*

b) Vergleiche die drei Ansätze entlang der Kriterien **Rechenkosten vor dem eigentlichen Lauf**, **Anpassungsfähigkeit während des Laufs** und **Allgemeinheit (auch außerhalb von EA nutzbar)**. Tabellarisch oder als Stichpunkte. *(6 P)*

c) Triff eine **begründete Wahl**: Düsenoptimierungen sind teuer in der Auswertung (CFD-Simulation). Welcher der drei Ansätze ist hier am riskantesten und warum? *(3 P)*

Lösung:

a) Mögliche Zuordnung (es gibt mehrere gültige Varianten):

- **Tuning** (μ): Vor dem eigentlichen Lauf werden mehrere kurze EA-Läufe mit unterschiedlichen $\mu \in \{20, 50, 100\}$ durchgeführt und derjenige Wert mit dem besten Endergebnis fest gewählt. Generischer Ansatz, funktioniert auch für Nicht-EA-Algorithmen.
- **Self-adaptation** (σ): σ wird Teil des Lösungsvektors $(\vec{x}, \sigma)$ und mitvererbt/mutiert. Selektion bevorzugt Individuen mit „passendem“ σ — der Hyperparameter wird also selbst evolviert. EA-spezifisch.
- **Parameter Control** (σ via 1/5-Rule): Heuristische Regel passt σ während des Laufs anhand der gemessenen Erfolgsrate an (Skript-Beispiel **1+1-ES, 1/5-Rule**).

b)

Kriterium	Tuning	Self-adaptation	Parameter Control
Vorab-Kosten	hoch (viele Probeläufe)	niedrig	niedrig
Anpassung im Lauf	nein (statisch)	ja (über Selektion)	ja (über Heuristik)
Allgemeinheit	hoch (auch andere Algos)	nur EA	nur EA-typische Regeln

c) **Tuning** ist hier am riskantesten: Es erfordert *zusätzliche* vollständige (oder zumindest hinreichend lange) EA-Läufe nur zur HP-Wahl, und jede einzelne Fitness-Auswertung ist eine teure CFD-Simulation. Das Budget verpufft, bevor die eigentliche Optimierung beginnt. Self-adaptation oder Parameter Control nutzen dasselbe Auswertungsbudget gleichzeitig für Lösung **und** HP-Anpassung und sind hier deutlich budgetschonender — was auch der historische Grund ist, warum gerade die ES-Familie (Self-adapt., 1/5-Rule, später CMA-ES) im Engineering-Design dominiert.

Aufgabe 5 — Pseudocode-Analyse: Fehler in einer 1+1-ES-Implementierung *(15 Punkte)*

Ein Studi schreibt folgenden Pseudocode für eine 1+1-ES (zu minimieren):

```
1  x  ← startSolution()
2  y  ← f(x)
3  σ  ← 1
4  G  ← []                      # Speicher
5  loop until stop:
6      x_next ← x + ε,  ε ~ N(0, σ)
7      y_next ← f(x_next)
8      if y_next > y:                      # (A)
9          G.append(1)
10         x ← x_next; y ← y_next
11     else:
12         G.append(0)
13     s ← sum(G) / len(G)                 # (B)
14     if s > 1/5: σ ← σ * 1.5
15     if s < 1/5: σ ← σ / 1.5             # (C)
```

a) Finde **drei** Fehler/Abweichungen vom Skript-Algorithmus und korrigiere sie jeweils präzise. *(9 P)*

b) Erläutere, was passiert, wenn der Code so wie oben (mit Fehler A) auf $f(\vec{x}) = \sum x_i^2$ läuft: in welche Richtung „treibt“ die Lösung und warum? *(3 P)*

c) Warum ist die fehlende Begrenzung von G (Punkt B) auch bei korrekter Vergleichsrichtung ein Problem für die Adaptionsdynamik? *(3 P)*

Lösung:

a) Drei Fehler:

1. **Zeile 8 (A)**: Bei Minimierung muss Erfolg `y_next < y` sein, nicht >. Fix:

```
if y_next < y:
```

1. **Zeile 13 (B)**: Skript sieht einen **gleitenden Speicher der Länge** g vor („Älteste Eintragung verwerfen“). G wächst hier unbegrenzt — die Erfolgsrate s wird dadurch über die gesamte Historie gemittelt. Fix: nach append zusätzlich

```
if len(G) > g: G.pop(0)
```

und g als Parameter einführen.

1. **Zeile 14/15 (C bzw. die Reihenfolge)**: Die beiden `if`-Zweige können sich kaskadieren — bei $s > 1/5$ wird σ erst $\cdot 1{,}5$, dann fällt es nicht in den zweiten Zweig (ok), aber: s darf nur ausgewertet werden, wenn G überhaupt voll ist (Skript: „Wenn G vollständig“). Fix: Anpassung nur wenn `len(G) == g`. Außerdem fehlt im Pseudocode der Klon-Schritt explizit (Zeile 6 mutiert direkt x statt einen Klon x_{next} — hier syntaktisch ok, aber im Skript ausdrücklich Schritt 2/3).

(Alternative gültige Beobachtungen: σ ist skalar statt komponentenweiser Vektor $\vec{\sigma}$; `else`-Zweig in Zeile 11 muss beim Misserfolg x unverändert lassen — was hier zwar passiert, aber gemeinsam mit Fehler A fatal wirkt.)

b) Mit Fehler A wird ein Schritt nur dann akzeptiert, wenn er **schlechter** ist. Die Sequenz der akzeptierten $\vec{x}$ wandert systematisch in Richtung *größerer* f-Werte — bei $\sum x_i^2$ also vom Optimum $\vec{0}$ **weg**, theoretisch unbegrenzt. Zusätzlich sieht die 1/5-Rule diese Misserfolg-Akzeptanz als „Erfolg“ und vergrößert σ, was die Divergenz beschleunigt.

c) Wenn G unbegrenzt wächst, mittelt s die *gesamte* bisherige Erfolgsrate. Frühe Generationen (anderes σ, andere Position im Suchraum) dominieren dauerhaft die Statistik, neue Information wirkt nur noch infinitesimal auf s. Damit „friert“ σ effektiv ein, die 1/5-Rule verliert ihre Reaktivität — genau das Gegenteil dessen, was eine *Online*-Adaption leisten soll.

Aufgabe 6 — SMBO als Designentscheidung *(15 Punkte)*

Ein Team trainiert ein großes neuronales Netz; ein einzelner Trainingslauf kostet 6 Stunden GPU-Zeit. Es sollen 4 Hyperparameter (Lernrate, Batchgröße, Dropout, Weight Decay) gefunden werden, die die Validierungs-Loss minimieren. Das Budget beträgt 80 Trainingsläufe.

a) Erkläre **SMBO** in eigenen Worten und nenne die drei laut Skript kombinierten Disziplinen. Warum passt SMBO hier besser als eine klassische 1+1-ES oder CMA-ES? *(5 P)*

b) Skizziere eine plausible **Gesamtstrategie** für die 80 Läufe: Wie viele Läufe sollten in eine erste Versuchsplanung (z. B. Latin Hypercube), wie viele in die surrogat-getriebene Suche? Begründe mit dem Argument „teure Zielfunktion“. *(5 P)*

c) **Transfer**: In der Düsen-Optimierung (Aufgabe 4) kostet eine CFD-Auswertung 90 Minuten. Wäre SMBO dort ebenfalls die natürliche Wahl, oder spricht etwas dagegen? Argumentiere mit dem Skript-Hinweis zu SMBO-Anwendungen. *(5 P)*

Lösung:

a) **SMBO (Surrogate Model-Based Optimization)** ersetzt die teure echte Zielfunktion f durch ein billiges, datengetriebenes Ersatzmodell $\hat{f}$ (typischerweise Gauß-Prozess, Random Forest o. Ä.). Die Optimierung wählt nicht direkt auf f, sondern auf $\hat{f}$ den nächsten vielversprechenden Punkt aus, wertet dort *das echte* f aus und aktualisiert $\hat{f}$. Skript-Bausteine: **Versuchsplanung + Statistik/ML + Optimierung**.

Passend hier, weil 1+1-ES und CMA-ES typischerweise *tausende* Auswertungen brauchen (1+1-ES: pro Generation eine, CMA-ES: λ pro Generation). Bei 80 Läufen Gesamtbudget reicht das für CMA-ES kaum für *eine Handvoll* Generationen mit sinnvoller Populationsgröße. SMBO ist explizit für *teure* Zielfunktionen gebaut und nutzt jede einzelne Auswertung über das Surrogat maximal aus.

b) Plausibel z. B.: **20 Läufe** initiale Versuchsplanung (Latin Hypercube über die 4 HP) zum Anlernen des Surrogats — genug Punkte, damit das Modell den 4-dimensionalen Raum grob abdeckt, aber nicht so viele, dass das Budget für die eigentliche Suche fehlt. Restliche **60 Läufe** surrogat-getrieben (Acquisition Function wie Expected Improvement). Begründung: Bei teurer Zielfunktion ist jede „verschwendete“ Auswertung doppelt teuer; das Surrogat lenkt die Suche genau auf die Punkte mit höchstem erwartetem Informationsgewinn — Versuchsplanung bezahlt den initialen Modellaufbau, danach trägt jede neue echte Auswertung sowohl zum Optimum **als auch** zur Modellverbesserung bei.

c) Ja, auch hier ist SMBO eine natürliche Wahl. Das Skript nennt explizit als Anwendung: **„ML-Tuning, Engineering Design (Auto, Aerospace)“** — die Düsenoptimierung fällt direkt in „Aerospace/Engineering Design“ mit teurer Auswertung (CFD). 90 min/Auswertung ist exakt das Regime, für das SMBO konstruiert wurde. Argument dagegen wäre höchstens, wenn extrem viele Designvariablen vorlägen (Surrogate skalieren in der Dimension schlechter); bei einer typischen parametrisierten Düse mit wenigen Geometrieparametern ist SMBO aber vorzuziehen, gerade gegenüber popula-

tionsbasierten EA, deren Auswertungs-Budget bei 90-min-Calls schnell aufgebraucht wäre.